

OBSTACLE DETECTION ROBOT

Major Project Report | IoT & Robotics Training Program

Autonomous Obstacle Detection and Avoidance using ESP32, HC-SR04 and L298N

Submitted By	Subodh Ukande
Roll Number	IOT-RS-T-14/JY-10912
Year	III Year
Institution	IPS Academy, Indore (RGPV Affiliated)
Project Type	Major Project
Training Program	IoT & Robotics Training Program INTERNS ELITE- July Batch 2026
Submission Deadline	15 September 2026

Prepared in accordance with the project submission guidelines provided by the program mentor.

ACKNOWLEDGEMENT

I sincerely thank the project mentor and the IoT & Robotics Training Program team for providing the project framework, technical guidance, and opportunity to work on an embedded robotics application. I also acknowledge the academic support provided by IPS Academy, Indore, which enabled me to apply concepts of microcontrollers, sensors, motor control, and programming in a practical project.

CONTENTS

1. Project Title and Objective	4
2. Project Description	4
3. Component List	5
4. System Block Diagram	5
5. Circuit Diagram and Circuit Explanation	6
6. Hardware Design	6
7. Software Implementation	7
8. System Workflow / Navigation Algorithm	7
9. Source Code	8
10. Testing Results	9
11. Challenges Faced	9
12. Conclusion	10
13. Learning Outcomes	10
14. Submission Checklist	10

1. PROJECT TITLE AND OBJECTIVE

Project Title

Obstacle Detection Robot using ESP32, HC-SR04 Ultrasonic Sensor and L298N Motor Driver

Objective

The objective of this project is to design and develop an autonomous Obstacle Detection Robot capable of continuously sensing obstacles in its surroundings and changing its movement direction to avoid collisions in real time. An ESP32 microcontroller processes distance information obtained from an HC-SR04 ultrasonic sensor and controls the DC geared motors through an L298N motor driver. A servo-mounted sensor can be used to scan different directions and select a safer path when an obstacle is detected.

The project demonstrates the practical integration of an embedded controller, ultrasonic sensing, servo positioning, differential motor control, and autonomous decision-making. The intended result is smooth movement, reliable obstacle detection, and effective collision avoidance.

Project Scope

- Real-time ultrasonic distance measurement.
- Automatic forward movement when the path is clear.
- Stopping or slowing when an obstacle is detected.
- Reverse motion for short recovery when necessary.
- Scanning left/right directions using a servo-mounted ultrasonic sensor.
- Selecting an alternate direction based on available distance.
- Serial Monitor feedback for sensor and robot status.

2. PROJECT DESCRIPTION

The robot uses the ESP32 as the central control unit. The HC-SR04 ultrasonic sensor emits an ultrasonic pulse and measures the echo return time to estimate the distance to an object. The sensor is mounted at the front of the robot and may be rotated by a servo to inspect different directions. The ESP32 compares the measured distance with a safety threshold and decides whether to move forward, stop, reverse, turn left, or turn right.

Motor commands are supplied to an L298N motor driver, which controls the direction and drive of the four DC geared motors. The mechanical chassis supports the motors and wheels, while a suitable battery provides power. The control logic is intentionally modular so that additional features such as Bluetooth, Wi-Fi, voice control, or IoT monitoring can be integrated later.

Figure 1. System Block Diagram

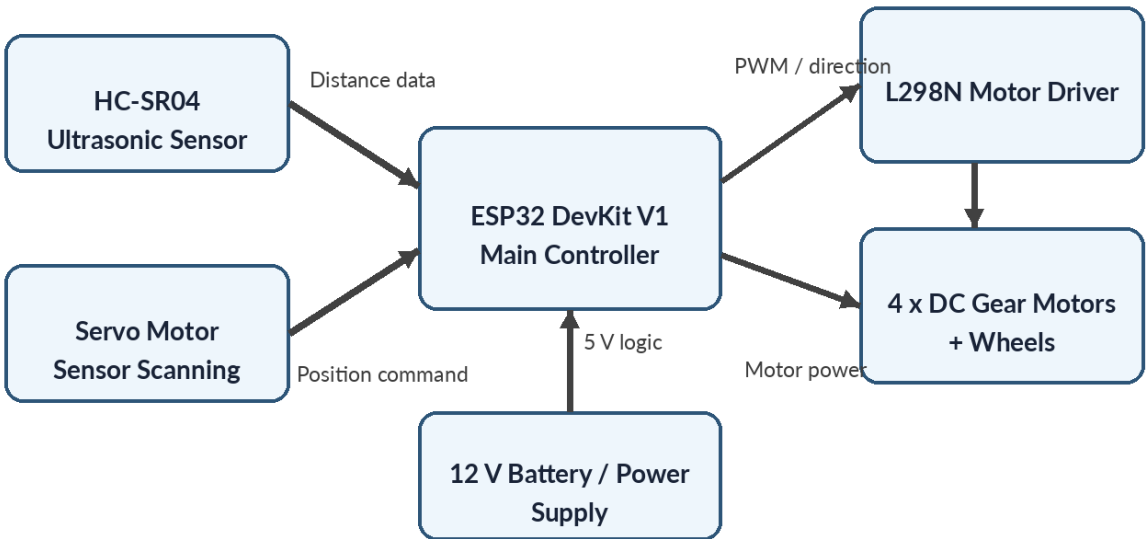


Figure 1. System block diagram of the proposed obstacle detection robot.

3. COMPONENT LIST

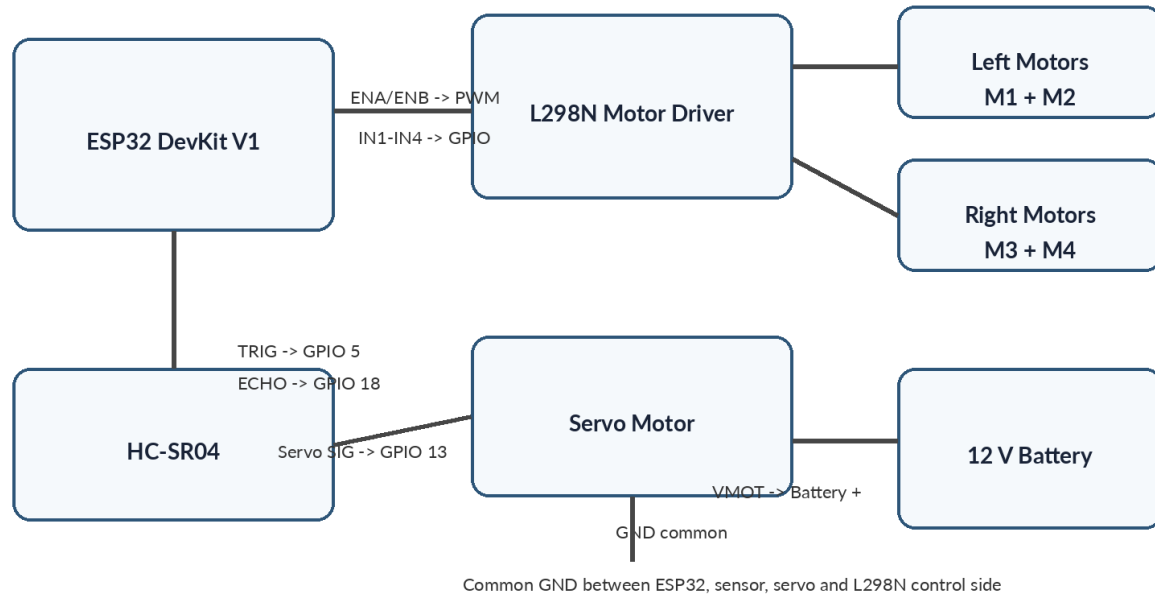
S.No.	Component	Typical Quantity	Purpose
1	ESP32 DevKit V1	1	Main controller and decision unit
2	HC-SR04 Ultrasonic Sensor	1	Obstacle distance measurement
3	Servo Motor	1	Rotates sensor for directional scanning
4	L298N Motor Driver	1	Drives DC motors and controls direction
5	DC TT Geared Motors	4	Robot propulsion
6	Robot Chassis + Wheels	1 set	Mechanical support and movement
7	Battery / Power Supply	1	Provides power to motors and system
8	Jumper Wires / Connectors	As required	Electrical interconnections

4. SYSTEM BLOCK DIAGRAM

The block diagram shows the flow of sensing, processing, and actuation. The ultrasonic sensor provides distance data to the ESP32. The ESP32 generates control commands for the L298N driver, while the driver applies the required motor direction and drive signals. The servo supports directional scanning of the sensor.

5. CIRCUIT DIAGRAM AND CIRCUIT EXPLANATION

Figure 2. Simplified Wiring Diagram



Note: Connect ESP32 logic to appropriate driver input pins used in the source code; verify pin mapping before powering the circuit.

Figure 2. Simplified wiring reference for the ESP32-based robot.

Circuit Explanation

- The ESP32 acts as the controller and provides logic signals to the motor driver and servo.
- The HC-SR04 TRIG pin is connected to a digital GPIO output and the ECHO pin to a digital GPIO input. The code converts echo time into distance.
- The servo signal wire is connected to a PWM-capable ESP32 GPIO. The servo rotates the sensor to predefined angles during obstacle scanning.
- The L298N receives motor-direction/control inputs from the ESP32. Its output channels drive the DC geared motors.
- The motor supply is taken from the battery through the L298N power stage. A common ground is maintained between the controller and driver logic side.
- Before final powering, the exact GPIO-to-driver connections must match the pin definitions in the uploaded source code and the motor polarity should be checked.

6. HARDWARE DESIGN

The robot is arranged as a four-wheel differential-drive platform. The four geared motors are mounted on the chassis, with the controller and motor driver placed above the main body to keep wiring compact. The ultrasonic sensor is positioned at the front and mounted on the servo so the sensing direction can be changed without rotating the whole chassis.

The design prioritizes a low center of gravity, short motor-current paths, secure connectors, and protected placement of the ESP32. The battery is positioned so that the vehicle remains stable during starting, stopping, and turning operations.

7. SOFTWARE IMPLEMENTATION

The software is written for the ESP32 using the Arduino development environment. The program is divided into reusable functions for distance measurement, motor control, forward/reverse movement, left/right turning, servo scanning, and decision-making.

Major Software Functions

- `readDistance()` – measures ultrasonic echo time and converts it to distance.
- `moveForward()` / `moveBackward()` – commands both motor groups.
- `turnLeft()` / `turnRight()` – creates differential movement for direction change.
- `stopMotors()` – immediately disables drive movement.
- `lookLeft()` / `lookRight()` – rotates the sensor through the servo.
- `chooseDirection()` – compares scan results and selects the safer direction.

Control Logic

A configurable obstacle threshold is used. When the measured front distance is above the threshold, the robot continues forward. When the distance falls below the threshold, the robot stops, reverses slightly if required, scans the left and right directions, and turns toward the side with the greater clear distance. The sensor then returns toward the forward position and the navigation cycle resumes.

8. SYSTEM WORKFLOW / NAVIGATION ALGORITHM

1. Initialize ESP32 GPIO pins, motor driver, ultrasonic sensor, and servo.
2. Set the ultrasonic sensor to its forward-facing position.
3. Measure the front obstacle distance.
4. If the path is clear, move forward.
5. If an obstacle is within the threshold, stop the motors.
6. Move backward briefly to create turning space, where required.
7. Rotate the sensor left and measure distance.
8. Rotate the sensor right and measure distance.
9. Compare the two measured distances.
10. Turn toward the side having the larger safe distance.
11. Return the sensor to the forward position.
12. Repeat the sensing and navigation cycle continuously.

9. SOURCE CODE

The complete ESP32 Arduino source code is provided as a separate .ino file with this submission. The firmware follows the modular structure described in Section 7 and implements ultrasonic distance measurement, motor control, servo scanning, and automatic direction selection.

For final demonstration, verify the GPIO assignments and motor polarity against the actual hardware before uploading. The submitted GitHub repository should contain the final tested source file and meaningful commit history.

11. CHALLENGES FACED

- Maintaining stable power for the controller and motors while keeping wiring secure.
- Avoiding false obstacle detection caused by sensor angle, surface type, or very short distances.
- Coordinating servo movement with sensor measurement so that the scan result is meaningful.
- Selecting motor timing and turn duration that produce repeatable turns on the actual chassis.
- Keeping the physical wiring consistent with the GPIO pin definitions used by the ESP32 program.

The practical solution is iterative testing: verify one subsystem at a time, confirm motor direction, calibrate the safe-distance threshold, tune scan delays and turn duration, then integrate the complete navigation cycle.

12. CONCLUSION

The Obstacle Detection Robot demonstrates how an ESP32 microcontroller can combine sensor input and motor actuation to create an autonomous embedded system. The project integrates ultrasonic ranging, servo-based directional sensing, motor-driver control, and rule-based navigation to reduce collision risk and select an alternate path when the forward route is blocked.

The project is suitable as a Major Project because it covers the core requirements of obstacle detection, continuous environmental monitoring, automatic stop/reverse/turn behavior, DC geared motors, an L298N-type motor driver, appropriate power, serial feedback, modular code, and practical testing. Additional capabilities such as Bluetooth, Wi-Fi, IoT monitoring, voice control, mapping, or camera-based vision can be added in later iterations.

13. LEARNING OUTCOMES

- ESP32 GPIO and PWM-based control.
- Ultrasonic sensor interfacing and distance calculation.
- Servo motor control and directional scanning.
- L298N motor-driver interfacing and differential steering.
- Embedded C/C++ programming using Arduino IDE.
- Modular firmware design and debugging through Serial Monitor.
- Basic autonomous navigation and obstacle-avoidance logic.
- Hardware wiring, testing, fault isolation, and system integration.
- Technical documentation and GitHub-based project submission practices.